PLP - Segundo Recuperatorio - 1er cuatrimestre de 2025

#Orden	Turno	Libreta	Apellido y Nombre	Ej1	Ej2	Ej3	Nota Final

Este examen se aprueba obteniendo al menos dos ejercicios bien (B) y uno regular (R), y se promociona con al menos dos ejercicios muy bien (MB) y uno bien (B). Es posible obtener una aprobación condicional con un ejercicio muy bien (MB), uno bien (B) y uno insuficiente (I), pero habiendo entregado algo que contribuya a la solución del ejercicio. Las notas para cada ejercicio son: -, I, R, B, MB, E. Entregar cada ejercicio en hojas separadas. Poner nombre, apellido y número de orden en todas las hojas, y numerarlas. Se puede utilizar todo lo definido en las prácticas y todo lo que se dio en clase, colocando referencias claras. El orden de los ejercicios es arbitrario. Recomendamos leer el parcial completo antes de empezar a resolverlo.

Ejercicio 1 - Programación Lógica

Implementar los predicados respetando en cada caso la instanciación pedida. Los generadores deben cubrir todas las instancias válidas de aquello que generan sin repetir dos veces la misma. Se deben indicar los patrones de instanciación de todos los predicados auxiliares. No usar macros que colapsen cláusulas, cut (!) ni predicados de alto orden, con la única excepción de not. **No está permitido el uso de estructuras auxiliares.**

a) Definir el predicado `unico(+L,-U)` que es verdadero cuando `U` aparece una sola vez en la lista `L`. Resolver sin contar apariciones. Por ejemplo:

```
?- unico([a,b,c,a,a,b,d],U).
U = c ;
U = d ;
false.
```

b) Definir el predicado `sinRepetidos(+L)` que represente la siguiente propiedad: *una lista L hace verdadero a sinRepetidos si todos sus elementos son únicos.*

c) En este inciso vamos a representar a un subconjunto de las fórmulas de lógica proposicional con negación e implicación. Los átomos utilizados serán `neg(P)` e `imp(P,Q)`, con `P` y `Q` fórmulas. Definir el predicado `formula(+VS,-F)` que instancia en `F` **todas** las fórmulas que usan las variables proposicionales que aparecen en la lista `VS`. Por ejemplo:

```
?- formula([p,q],F).
F = p;
F = q;
F = neg(p);
F = neg(q);
```

```
F = neg(neg(p));
F = neg(neg(q));
F = imp(p,p);
F = imp(p,q);
...
```

```
F = neg(imp(q,q));
F = imp(p,neg(p));
...
F = imp(neg(q),imp(neg(p),p));
...
```

Sugerencia: prestar atención al orden en que se devuelven los resultados. No es necesario que se mantenga el orden del ejemplo, pero sí que se instancien todas las fórmulas.

d) ¿El predicado anterior es reversible en `F`? Justificar brevemente.

Ejercicio 2 - Resolución

Consideremos el siguiente lenguaje del dominio de funciones numéricas:

- $X \leq Y$: predicado binario que representa que X es menor o igual que Y
- $cota(F, X)$: predicado binario que representa que X es una cota superior de la función F
- $sup(F)$: símbolo de función unario que representa el supremo de la función F
- $ev(F, N)$: símbolo de función binario que representa el resultado de evaluar la función F en N

Sean los siguientes axiomas:

- $\forall F.\forall X.(cota(F, X) \Leftrightarrow \forall N.ev(F, N) \leq X)$
  (definición de cota superior)
- $\forall F.(cota(F, sup(F)) \wedge \forall Y.(cota(F, Y) \Rightarrow sup(F) \leq Y))$
  (el supremo de una función es una cota superior, y es la más chica de las cotas superiores)
- $\forall X.\forall Y.\forall Z.((X \leq Y \wedge Y \leq Z) \Rightarrow X \leq Z)$
  (transitividad de la relación menor o igual)

Se pide:

a. Pasar a forma clausal cada fórmula

b. Demostrar usando resolución que si una función es más chica que otra en todo punto, el supremo también es más chico:
$\forall F.\forall G.((\forall N.ev(F,N) \leq ev(G,N)) \Rightarrow sup(F) \leq sup(G))$

c. La resolución utilizada en el punto anterior ¿fue SLD? Justificar.

Ejercicio 3 - Inferencia, Deducción Natural y POO

a) Consideremos el Cálculo Lambda tipado extendido con intervalos de números naturales:

$$\sigma ::= \ldots \mid \mathsf{Intervalo}$$
$$M ::= \ldots \mid [M,M] \mid \text{case } M \text{ of } [\,] \rightsquigarrow M; x :: y \rightsquigarrow M$$

donde $[M,N]$ es el intervalo que va desde el número M hasta N (inclusive) y case M of $[\,] \rightsquigarrow N; x :: y \rightsquigarrow O$ es un observador que recorre el intervalo M como si fuese una lista: con un caso para el intervalo vacío (N) y otro para un intervalo con al menos un elemento (O). En el segundo caso, las apariciones libres de la variable x en O se ligarán al primer elemento del intervalo y las apariciones libres de y en O se ligarán al intervalo sin su primer elemento.

Se extiende también el algoritmo de inferencia de la siguiente manera:

$$\begin{array}{l} \mathcal{I}(\Gamma \mid [M_1,M_2]) = (\mathsf{Intervalo} \mid \{\tau_1 \stackrel{?}{=} Nat, \tau_1 \stackrel{?}{=} \tau_2\} \cup E_1 \cup E_2) \\ \quad \text{donde} \quad \mathcal{I}(\Gamma \mid M_1) = (\tau_1 \mid E_1) \\ \qquad\qquad\quad \mathcal{I}(\Gamma \mid M_2) = (\tau_2 \mid E_2) \end{array}$$

$$\begin{array}{l} \mathcal{I}(\Gamma \mid \text{case } M_1 \text{ of } [\,] \rightsquigarrow M_2; x :: y \rightsquigarrow M_3) = \\ (\tau_2 \mid \{\tau_2 \stackrel{?}{=} \tau_3, \tau_1 \stackrel{?}{=} \mathsf{Intervalo}, X_x \stackrel{?}{=} Nat, X_y \stackrel{?}{=} \mathsf{Intervalo}\} \cup E_1 \cup E_2 \cup E_3) \\ \quad \text{donde} \quad \mathcal{I}(\Gamma \mid M_1) = (\tau_1 \mid E_1) \\ \qquad\qquad\quad \mathcal{I}(\Gamma \mid M_2) = (\tau_2 \mid E_2) \\ \qquad\qquad\quad \mathcal{I}(\Gamma, x : X_x, y : X_y \mid M_3) = (\tau_3 \mid E_3) \\ \qquad\qquad\quad X_x \text{ y } X_y \text{ variables frescas} \end{array}$$

Usar el algoritmo $\mathcal{I}$ para inferir el tipo de la siguiente expresión, o demostrar que no es tipable:

$$\text{case } (\lambda x.[\underline{1}, x])\ y \text{ of } [\,] \rightsquigarrow (\lambda z.[y,y]); x :: z \rightsquigarrow z$$

b) Demostrar el siguiente teorema usando deducción natural, sin utilizar principios clásicos:

$$(\forall X.\exists Y.(P(X,Y) \wedge Q(Y,X))) \Rightarrow \exists X.\exists Y.Q(Y,X)$$

c) En una empresa, al final del día se realizan varias tareas rutinarias sobre distintas estructuras de datos. Estas tareas se encapsulan en bloques de código y deben ejecutarse secuencialmente sobre un mismo objeto. Se desea implementar un mecanismo genérico que permita realizar esta secuencia de operaciones sobre cualquier objeto.
Implementar en la clase `Object` el método `runBatch`, que reciba como parámetro una colección de bloques. Cada bloque debe tomar el objeto receptor como argumento, y se debe ejecutar en orden. El método debe devolver el resultado de la ejecución del último bloque.
Por ejemplo:

```
| lista resultado |
lista := OrderedCollection new.
resultado := lista runBatch: #(
    [ :obj | obj add: 1 ]
    [ :obj | obj add: 2 ]
    [ :obj | 'último paso' ])
```

Al final de la ejecución, `resultado` es `'último paso'`, mientras que `lista` contiene 1 y 2.